Rithmomachia Game Script Manual (Godot GDScript)

This manual provides documentation for the core logic functions across the game's scripts.

I. Board Management and Initialization (board.gd)

These functions handle setting up the game environment, generating the board visually, and preparing pieces.

- **_parse_piece_data(id_string: String) -> Dictionary**
  - **Purpose:** Takes a piece ID string (e.g., "S289_1" or "c006_1") and converts it into a structured dictionary of data.
  - **Details:** Determines the piece's color (capital letter is white, lowercase is black), shape (S, T, C, P), and its numerical label (value). This is crucial for initializing the Piece.gd script.
- **_ready()**
  - **Purpose:** The standard Godot initialization function, called when the node enters the scene tree.
  - **Details:** It initializes the move_validator, calls functions to build the visual board and labels, sets up the captured piece displays, creates the Phase and Undo buttons, and shows the initial game setup window. This is the entry point for game creation.
- **create_game_setup_window()**
  - **Purpose:** Programmatically creates and displays the setup window where players select the victory thresholds (N0 for pieces, N1 for total value).
  - **Details:** This window is critical for setting the "difficulty" or game type (Short, Medium, Long) by assigning values to victory_n0 and victory_n1.
- **execute_move(from_tile: ColorRect, to_tile: ColorRect)**
  - **Purpose:** Physically moves the selected piece from one tile to another on the board after the move has been validated.
  - **Details:** It updates the internal initial_board_state array, removes the piece from the starting tile, adds it to the destination tile, and logs the move using log_move().
- **generate_board()**
  - **Purpose:** Creates the 8x16 grid of ColorRect nodes (the tiles) that form the game board.
  - **Details:** Each generated tile is assigned the tile.gd script and connected to the _on_tile_clicked function for player input.
- **get_tile_at_coords(x: int, y: int) -> ColorRect**
  - **Purpose:** A utility function to retrieve a specific tile node based on its X (column) and Y (row) coordinates.
  - **Details:** This is essential for the move_validator and victory_checker scripts to check the contents of any specific square on the board.
- **setup_pieces()**
  - **Purpose:** Reads the hardcoded initial_board_state array to place all game pieces onto the board at the start of the game.

    - **Details:** It iterates through the state array, instantiates the piece_scene, and calls the tile's place_piece function for proper placement and initialization.

II. Turn Flow and Logging (board.gd)

These functions manage the current player, the three phases of a turn, and the recording of game events for the log/undo system.

- **advance_to_move_phase()**
    - **Purpose:** Changes the game state from PRE_MOVE_CAPTURE to MOVE phase.
    - **Details:** This function is typically triggered by the player pressing the "Done Capturing - Ready to Move" button. It disables capturing and enables a single movement for the current player.
- **advance_to_next_player()**
    - **Purpose:** Completes the current player's turn and transitions control to the opposing player.
    - **Details:** It calls finalize_turn_log() to save the entire turn, switches current_player, resets the phase to PRE_MOVE_CAPTURE, and checks if the new player has any legal moves to prevent a blocked loss.
- **capture_board_state() -> Dictionary**
    - **Purpose:** Saves a complete snapshot of the game's current configuration.
    - **Details:** This function is vital for the **Undo** feature. It records the position and internal state (especially the remaining values of Pyramids) of every piece on the board.
- **check_if_player_can_move() -> bool**
    - **Purpose:** Determines if the current player has any legal non-capture moves available.
    - **Details:** It uses the move_validator to check for moves and, if none are found, calls show_game_over() to declare the opponent the winner by blockade.
- **declare_victory(winner: String, victory_type: String)**
    - **Purpose:** Executes the game-ending sequence when a victory condition (Common or Proper) is met.
    - **Details:** It sets the game_ended flag to true, updates the UI with the victory message, and critically, calls finalize_turn_log() and the log export function to ensure the final state is recorded.
- **log_capture(attacker_piece, attacker_pos, victim_piece, ...)**
    - **Purpose:** Records the full details of a piece or sub-piece capture in the current_turn_log.
    - **Details:** It stores information about the captured piece, the capturing piece(s), the capture type (e.g., "sum", "divisor", "equality"), and whether a helper piece was involved. This log is used for reporting and the Undo feature.
- **log_move(piece: Node2D, from_pos: Vector2i, to_pos: Vector2i)**
    - **Purpose:** Records the movement of a piece in the current_turn_log.
    - **Details:** Stores the moving piece's ID, its value, and the start/end coordinates of the move.

- **restore_board_state(state: Dictionary)**
  - **Purpose:** Clears the board and recreates all pieces and their states based on a previously saved state dictionary.
  - **Details:** This is the core logic of the **Undo** function. It removes all nodes, clears the captured zones, restores the initial_board_state array, and reinstantiates all pieces in their previous positions and with their correct internal values (especially for Pyramids).
- **start_new_turn_log()**
  - **Purpose:** Prepares the current_turn_log dictionary at the beginning of a new turn.
  - **Details: Crucially, it immediately calls capture_board_state()** to take a snapshot of the board *before* the current player makes any changes. This snapshot is what is used if the player later chooses to undo the turn.
- **undo_last_turn()**
  - **Purpose:** Reverts the game state back to the beginning of the most recently completed turn.
  - **Details:** It pops the last log entry from game_log, restores the board using the saved board_state_snapshot, resets current_player and current_phase, and updates all UI elements.

III. Piece Capture and Pyramid Logic (board.gd, piece.gd)

These functions handle the execution and tracking of capturing pieces, including the complex logic for Pyramids (which are stacks of sub-pieces).

- **attempt_capture(attacker_tile: ColorRect, victim_tile: ColorRect)**
  - **Purpose:** The primary controller function that is called when a player clicks an attacker and then a target during a capture phase.
  - **Details:** It consults the move_validator to confirm the capture is legal, selects the first valid capture type found (e.g., sum, multiple), logs the event, and then calls the appropriate execution function based on whether the target is a Pyramid or a regular piece.
- **execute_full_piece_removal(victim_tile: ColorRect)**
  - **Purpose:** Removes a non-Pyramid piece (Circle, Triangle, Square) from the board entirely.
  - **Details:** It logs the value of the piece to the capturing player's score, removes the piece node from the tile, moves it to the captured zone display, and updates the board array.
- **execute_pyramid_subpiece_removal(pyramid_piece: Node2D, captured_value: int, pyramid_pos: Vector2i)**
  - **Purpose:** Handles the specific case of capturing a single numerical sub-piece from an opponent's Pyramid.
  - **Details:** It calls pyramid_piece.remove_subpiece(captured_value) to update the piece's internal data, adds the captured_value to the player's score, updates the Pyramid's visual label, and removes the entire Pyramid if no sub-pieces remain.

- **on_piece_captured(piece_color: String, piece_value: int)**
  - **Purpose:** Central handler for updating the captured piece count and total value for the player who performed the capture.
  - **Details:** It determines which player captured which color, increments the piece count and value, updates the UI displays, and immediately calls the victory checking functions (check_victory_by_body and check_victory_by_goods) to see if the common victory condition has been met.
- **remove_subpiece(value_to_remove: int)** (in piece.gd)
  - **Purpose:** Removes a single captured value from a Pyramid piece's internal data.
  - **Details:** This function is inside the Piece.gd script. It finds the value within the Pyramid's piece_label array and removes it, then calls update_label_display() to ensure the Pyramid's visual text label reflects its new, lower value.

IV. Move and Capture Validation (move_validator.gd)

These functions implement the core rules of movement and the mathematical capture relationships of Rithmomachia.

- **check_sum_and_difference(captor_pos, captor_val, target_pos, target_val) -> Array**
  - **Purpose:** Checks for the two-piece captures: Sum (A + B = Target) and Difference (|A - B| = Target).
  - **Details:** It iterates through all friendly pieces on the board, treating them as potential "helper" pieces (B). If the helper piece can also reach the target in one move, it tests the Sum and Difference relationships and returns an array of valid capture types, including the helper's position.
- **get_all_possible_captures(captor_pos: Vector2i, captor_piece: Node2D) -> Array**
  - **Purpose:** Finds every enemy piece on the board and tests *all* possible capture types against them using the currently selected captor piece.
  - **Details:** This function accounts for Pyramids, testing captures against each of the Pyramid's individual sub-values. It aggregates all possible capture results and is the main function used by the board when a piece is selected during a capture phase.
- **get_path_clear_and_distance(start_pos: Vector2i, end_pos: Vector2i) -> Dictionary**
  - **Purpose:** Calculates the number of empty squares between two orthogonally aligned pieces (used for Multiplier and Divisor captures).
  - **Details:** It checks every square between the start and end positions. If any square is occupied, the path is considered blocked ("clear": false). The distance is the count of *empty* tiles between the pieces.
- **get_potential_moves(piece: Node2D, pos: Vector2i) -> Array[Vector2i]**
  - **Purpose:** Returns the theoretical destination squares a piece can move to based on its shape, *without* checking for obstacles or if the destination is empty.
  - **Details:** Circles move 1 step, Triangles 2, and Squares/Pyramids 3, all only orthogonally.
- **get_valid_moves(piece: Node2D, from_pos: Vector2i) -> Array[Vector2i]**
  - **Purpose:** Filters the potential moves to only include those that land on an empty

square and where the path is completely clear of other pieces.
    - **Details:** This function is the primary method used to highlight where a selected piece can legally move during the MOVE phase.
- **is_move_valid(piece: Node2D, from_pos: Vector2i, to_pos: Vector2i) -> bool**
    - **Purpose:** The core check for whether a straight movement is legal.
    - **Details:** Ensures the destination is empty and verifies that the correct number of intermediate squares are empty, corresponding to the piece's fixed movement distance (1, 2, or 3 spaces).
- **is_within_capture_range(piece: Node2D, from_pos: Vector2i, to_pos: Vector2i) -> bool**
    - **Purpose:** Checks if a target piece is exactly one move away (1, 2, or 3 squares, depending on the piece) and that the path to it is clear.
    - **Details:** This is used for Equality, Sum, and Difference captures, where the captor(s) must be able to move directly onto the target square.

V. Victory Condition Checking (victory_checker.gd)

These functions implement the Proper Victory conditions, which require mathematical progressions in the enemy's territory.

- **check_arithmetical_victory(color: String) -> Dictionary**
    - **Purpose:** Checks if the player has three pieces in the enemy's territory forming an arithmetic progression (e.g., (a, a+d, a+2d) with common difference d, for example: 3, 5, 7).
    - **Details:** It finds all the player's pieces in the opponent's starting half, checks every possible combination of three pieces, sorts their values, and tests the arithmetic relationship.
- **check_for_win(current_color: String, captured_pieces: Array) -> Dictionary**
    - **Purpose:** The main entry point to check all victory conditions for the current player at the end of their turn.
    - **Details:** It sequentially checks for all four victory types: Arithmetic, Geometric, Harmonic, and the Common Victories (Body and Goods), returning immediately upon finding the first win condition.
- **check_geometrical_victory(color: String) -> Dictionary**
    - **Purpose:** Checks if the player has three pieces in the enemy's territory forming a geometric progression (e.g., (a, ar, ar^2) with ratio r>0, for example: 2, 4, 8).
    - **Details:** Similar to the arithmetic check, it iterates through all combinations of three pieces in the enemy half and verifies the geometric relationship.
- **check_harmonic_victory(color: String) -> Dictionary**
    - **Purpose:** Checks if the player has three pieces in the enemy's territory forming a harmonic progression (e.g., (1/a, 1/b, 1/c) form an arithmetic progression, for example: 9, 15, 45).
    - **Details:** This is the most complex of the progressions, requiring the distinct three values in the enemy's territory to satisfy the formula derived from the definition.

- **get_pieces_in_enemy_territory(color: String) -> Array**
  - **Purpose:** A helper function to gather all pieces belonging to the specified player that have successfully crossed into the opponent's starting half of the board.
  - **Details:** For White, this means checking columns 8-15; for Black, it checks columns 0-7. This limits the scope of the Proper Victory checks.